

SECOS System Sub-Module Design

Component Specification: Machine Learning Engine

Module Name	Machine Learning Engine
Domain Responsibility	Time-Series Forecasting & Energy Prediction
Integration Phase	Phase 2 (Daily Forecasting & Tier Alerts)
Target KPI	>85% Prediction Accuracy

1. Architecture & Data Flow

Design Philosophy: The ML Engine operates as a stateless, isolated microservice. It receives no direct user interaction and contains no business logic regarding electricity pricing or Egyptian tariff tiers. Its sole responsibility is accurately mapping environmental and behavioral variables to future kilowatt-hour (kWh) consumption.

During Phase 2 (Daily Forecasting), the SECOS Backend orchestrates a daily cron job that pushes data to the ML Engine. The internal data pipeline follows a strict sequence:

- Data Ingestion:** The JSON payload is parsed to extract `weather_forecast` (temperature, humidity, conditions) and `historical_consumption` lags alongside the user's `calibrated_multiplier`.
- Feature Engineering:** Data is cleaned, missing weather values are handled, and numerical features such as `square_footage` and `household_size` are normalized and scaled.
- Model Routing:** Engineered features are passed into supervised learning models based on the required prediction complexity.
- Output Generation:** The mathematical predictions are aggregated and formatted back into the required JSON array format for the Backend.

2. API Contract Specifications

The module strictly communicates via standard JSON REST payloads. The Backend triggers the process using the following endpoint.

Endpoint: POST `/api/v1/ml/predict`

2.1 Request Payload (Backend → ML Engine)

The input must contain metadata, historical usage, and a daily weather forecast.

```
{ "user_id": "usr_98231", "timestamp": "2026-09-18T15:30:00Z", "forecast_days":
```

2.2 Response Payload (ML Engine → Backend)

The engine responds with a success status, the active model version, and an array of predicted kWh values matching the requested forecast days.

```
{ "status": "success", "model_version": "v1.2-linear-regression", "predictions":  
[ {"date": "2026-09-19", "predicted_kwh": 14.8}, {"date": "2026-09-20",  
"predicted_kwh": 18.2}, {"date": "2026-09-21", "predicted_kwh": 13.5}, {"date": "2026-  
09-22", "predicted_kwh": 14.1}, {"date": "2026-09-23", "predicted_kwh": 15.0}, {"date":  
"2026-09-24", "predicted_kwh": 13.9}, {"date": "2026-09-25", "predicted_kwh": 14.2} ] }
```

3. Model Selection & Inference Strategy

The system utilizes supervised machine learning algorithms to process the engineered features. The development timeline employs a phased rollout of models to ensure baseline stability before introducing complexity.

- **Phase A (Baseline): Linear Regression.** Deployed as the initial model (version v1.2-linear-regression). This provides a foundational, fast-executing prediction of basic energy values based on direct linear relationships (e.g., higher temperature correlates to higher usage).
- **Phase B (Advanced): Random Forest.** Implemented to capture complex, non-linear forecast patterns. By utilizing ensemble decision trees, this model will better account for behavioral nuances (like weekend vs. weekday usage shifts combined with specific weather thresholds) to deliver improved accuracy.

4. Evaluation Metrics & Testing Plan

To validate the ML Engine's effectiveness against the historical energy consumption datasets gathered during Weeks 1-3, structured evaluation will occur throughout the Weeks 4-10 development window.

Primary KPI Target: The system must achieve a Prediction Accuracy of > **85%** prior to integration in Week 10.

Testing Methodology:

- **Data Splitting:** Historical datasets will be divided into structured training (80%) and testing (20%) sets to prevent model overfitting.
- **Error Tracking:** Technical performance will be measured using standard regression metrics, specifically Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), to quantify the average deviation (in kWh) between predicted and actual consumption.
- **Milestone Validation:** The Random Forest model must consistently demonstrate a lower MAE than the Linear Regression baseline across cross-validated test folds before being promoted to the primary inference router.